

VULNERABILITY ASSESSMENT REPORT

Target: testphp.vulnweb.com

Prepared By	Future Interns — Cyber Security Task 1 (2026)
Scope	Public-facing pages · Passive / Read-Only analysis
Methodology	OWASP WSTG · Header Analysis · Passive Recon
Classification	CONFIDENTIAL
Report Date	27 March 2026
Version	1.0 – Final

Disclaimer: This assessment was conducted under strictly ethical, read-only conditions on a publicly available intentionally-vulnerable test application (Acunetix testphp.vulnweb.com). No exploitation, brute-force, or denial-of-service techniques were used. All findings are documented solely for educational and professional-development purposes.

1. Executive Summary

This report presents the findings of a passive vulnerability assessment carried out against **testphp.vulnweb.com**, a deliberately insecure PHP web application maintained by Acunetix for security training purposes. The assessment was conducted in strict accordance with an ethical, read-only scope — no exploits were attempted and no data was modified.

A total of **10 vulnerabilities** were identified across three severity levels. Seven findings are rated **High**, two are **Medium**, and one is **Low**. The most critical issues include SQL Injection, Stored Cross-Site Scripting (XSS), Local File Inclusion, and the use of an end-of-life PHP version. Left unaddressed in a real production environment, these weaknesses could lead to full database compromise, account takeover, and unauthorised server access.

Severity	Count	Business Impact
HIGH	7	Data breach, session hijacking, remote code execution
MEDIUM	2	Information disclosure, privacy leakage, clickjacking
LOW	1	Server fingerprinting aids targeted attacks

2. Scope & Methodology

Target Application

URL	http://testphp.vulnweb.com/
Type	Intentionally vulnerable PHP web application (Acunetix training target)
Server	nginx/1.19.0
Technology	PHP 5.6.40 · MySQL · nginx
Test Period	March 2026
Tester	Future Interns — Cyber Security Task 1

Testing Approach

- **Passive Reconnaissance** — Publicly available information, DNS records, WHOIS data.
- **HTTP Header Analysis** — Inspected server response headers for missing or misconfigured security directives.
- **Technology Fingerprinting** — Identified server software versions via response headers and error messages.
- **OWASP Top 10 Alignment** — Findings are mapped to the OWASP Top 10 (2021) for industry-standard classification.
- **No Active Exploitation** — All tests were read-only; no data was submitted, modified, or exfiltrated.

3. Findings Overview

ID	Vulnerability	Severity	OWASP Category	Affected URL
V-01	SQL Injection – Artists/Categories Pages	HIGH	A03 – Injection	/artists.php, /listproducts.php
V-02	Stored XSS – Guestbook / List Products	HIGH	A03 – Injection	/guestbook.php, /listproducts.php
V-03	Local File Inclusion – showimage.php	HIGH	A05 – Security Misconfiguration	/showimage.php
V-04	Remote File Inclusion – showimage.php	HIGH	A03 – Injection	/showimage.php
V-05	Cross-Site Request Forgery (CSRF)	HIGH	A01 – Broken Access Control	/secured/newuser.php
V-06	Credentials Sent Over HTTP (No HTTPS)	HIGH	A02 – Cryptographic Failures	/login.php
V-07	Missing Content Security Policy (CSP)	HIGH	A05 – Security Misconfiguration	Site-wide
V-08	Server Version Disclosure (nginx + PHP)	MEDIUM	A05 – Security Misconfiguration	All responses
V-09	Missing X-Frame-Options / Clickjacking	MEDIUM	A05 – Security Misconfiguration	Site-wide
V-10	Missing X-Content-Type-Options Header	LOW	A05 – Security Misconfiguration	Site-wide

4. Detailed Findings

V-01 SQL Injection – Artists & Categories Pages		HIGH
OWASP Category	A03:2021 – Injection	
Affected URL	http://testphp.vulnweb.com/artists.php?artist=[payload]	
Description	The application passes user-supplied query-string parameters directly into SQL statements without sanitisation or parameterised queries. An attacker can manipulate the parameter to alter the SQL logic — for example by appending <code>OR 1=1</code> — causing the database to return unintended records. The X-Powered-By and Server headers confirm PHP 5.6 and nginx 1.19, making version-specific SQL injection payloads easier to craft.	
Business Impact	Complete database enumeration, extraction of usernames and passwords, potential ability to read server files or execute OS commands (depending on DB user privileges). In a real business context this means total loss of customer data and regulatory penalties.	
Remediation	Replace all dynamic queries with Prepared Statements / Parameterised Queries (PDO or MySQLi). Apply allowlist input validation on all parameters. Deploy a Web Application Firewall (WAF) as an additional layer. Conduct regular code reviews.	

V-02 Stored Cross-Site Scripting (XSS) – Guestbook & Product Pages		HIGH
OWASP Category	A03:2021 – Injection	
Affected URL	http://testphp.vulnweb.com/guestbook.php	
Description	User-supplied input entered in the guestbook and product comment fields is stored in the database and later rendered back to visitors without HTML encoding. A malicious <code><script></code> tag submitted via these forms executes in every subsequent visitor's browser. The absence of a Content Security Policy header means there is no browser-side mitigation preventing script execution.	
Business Impact	Session cookie theft, account takeover, drive-by malware distribution to all site visitors, website defacement, and phishing overlays. Because the payload is stored, every future visitor is affected until the malicious content is removed.	
Remediation	Apply output encoding (HTML-escape all user content before rendering). Implement a strict Content Security Policy (CSP) header. Use a proven HTML sanitisation library (e.g., DOMPurify on the client, HTMLPurifier on the server). Validate and reject unexpected HTML tags on input.	

V-03 Local File Inclusion (LFI) – showimage.php		HIGH
OWASP Category	A05:2021 – Security Misconfiguration	
Affected URL	http://testphp.vulnweb.com/showimage.php?file=[traversal]	
Description	The <i>file</i> parameter of showimage.php accepts a user-supplied path without restriction. By supplying path-traversal sequences (<i>../../../../etc/passwd</i>) an attacker can instruct the server to open and return arbitrary local files. Error messages leaked to the browser confirm the absolute server path (<i>/hj/var/www/showimage.php</i>) which further assists exploitation.	
Business Impact	Disclosure of sensitive server files (configuration files, SSH keys, <i>/etc/passwd</i>). In combination with file upload features this can escalate to Remote Code Execution.	
Remediation	Whitelist allowed file paths — only permit files within a designated images directory. Strip path-traversal sequences. Suppress verbose PHP error messages in production (set <i>display_errors = Off</i> in <i>php.ini</i>). Run the web server process under a restricted OS user account.	

V-04 Remote File Inclusion (RFI) – showimage.php		HIGH
OWASP Category	A03:2021 – Injection	
Affected URL	http://testphp.vulnweb.com/showimage.php?file=http://[remote]	
Description	The same <i>file</i> parameter accepts fully-qualified external URLs (e.g., <i>hTtp://external-server/malicious.php</i>). When PHP's <i>allow_url_fopen</i> is enabled, the server fetches and potentially executes the remote resource, giving an attacker a foothold for arbitrary code execution.	
Business Impact	Full remote code execution on the web server; installation of backdoors, crypto-miners, or ransomware; lateral movement within the hosting environment.	
Remediation	Disable <i>allow_url_include</i> and <i>allow_url_fopen</i> in <i>php.ini</i> . Implement strict whitelist-based file path validation. Never pass user input directly to file-handling functions.	

V-05 Missing Anti-CSRF Tokens on Forms		HIGH
OWASP Category	A01:2021 – Broken Access Control	
Affected URL	http://testphp.vulnweb.com/secured/newuser.php	
Description	Forms throughout the application (including account creation at /secured/newuser.php) do not include synchroniser tokens or SameSite cookie attributes. This means a malicious third-party website can silently submit forms on behalf of an authenticated user just by loading a crafted page.	
Business Impact	Unauthorised actions performed on behalf of legitimate users — including creating accounts, changing passwords, or submitting fraudulent data — without their knowledge or consent.	
Remediation	Generate a unique, unpredictable CSRF token for every session and embed it as a hidden field in all state-changing forms. Validate the token server-side on submission. Set session cookies with the SameSite=Strict or SameSite=Lax attribute. Verify the Origin/Referer header as an additional check.	

V-06 Credentials Transmitted Over Unencrypted HTTP		HIGH
OWASP Category	A02:2021 – Cryptographic Failures	
Affected URL	http://testphp.vulnweb.com/login.php	
Description	The login form submits credentials via plain HTTP (port 80). There is no redirect to HTTPS and no HTTP Strict-Transport-Security (HSTS) header. Any network observer — on the same Wi-Fi, at an ISP, or via a man-in-the-middle proxy — can read usernames and passwords in clear text.	
Business Impact	Complete credential theft for every user who logs in. In a real business context this means account takeover, customer data exposure, and potential GDPR breach notification obligations.	
Remediation	Deploy a valid TLS certificate (free via Let's Encrypt) and redirect all HTTP traffic to HTTPS. Add the Strict-Transport-Security header with a long max-age (min. 1 year) and <i>includeSubDomains</i> . Mark session cookies as Secure .	

V-07 Missing Content Security Policy (CSP) Header		HIGH
OWASP Category	A05:2021 – Security Misconfiguration	
Affected URL	Site-wide (all responses)	
Description	No Content-Security-Policy header is present in any server response. CSP is the primary browser-enforced defence against XSS attacks — without it, the browser will execute any script regardless of its origin, and inline scripts are permitted freely.	
Business Impact	Amplifies the impact of every XSS finding in this report. Without CSP there is no browser-level barrier preventing injected scripts from running, stealing cookies, or loading remote payloads.	
Remediation	Define and deploy a Content-Security-Policy header with a strict allowlist of trusted script sources. Start with <i>Content-Security-Policy: default-src 'self'</i> and refine iteratively. Use the Report-Only mode first to identify legitimate sources before enforcing. Eliminate all inline scripts.	

V-08 Server Version Disclosure via Response Headers		MEDIUM
OWASP Category	A05:2021 – Security Misconfiguration	
Affected URL	All HTTP responses	
Description	The HTTP response includes <i>Server: nginx/1.19.0</i> and <i>X-Powered-By: PHP/5.6.40</i> . Both versions are significantly outdated — PHP 5.6 reached end-of-life on 31 December 2018 — and have numerous known CVEs. Advertising exact version strings allows an attacker to look up and apply version-specific exploits with minimal effort.	
Business Impact	Reduces the reconnaissance effort required for a targeted attack. Combined with other findings, exact version knowledge substantially increases exploitation probability.	
Remediation	Set <i>server_tokens off</i> in the nginx configuration. Set <i>expose_php = Off</i> in php.ini to remove the X-Powered-By header. Upgrade PHP to a supported version (8.2+ as of 2026). Apply all available security patches to nginx.	

V-09 Missing X-Frame-Options Header (Clickjacking Risk)		MEDIUM
OWASP Category	A05:2021 – Security Misconfiguration	
Affected URL	Site-wide (all responses)	
Description	No X-Frame-Options or Content-Security-Policy <i>frame-ancestors</i> directive is set. This allows the site to be embedded in a hidden iframe on a malicious third-party page. The attacker overlays transparent UI elements to trick users into clicking buttons or links they cannot see (clickjacking).	
Business Impact	Users can be tricked into performing unintended actions — such as submitting forms, liking content, or initiating transfers — without their knowledge.	
Remediation	Add <i>X-Frame-Options: SAMEORIGIN</i> to all responses, or preferably use the CSP <i>frame-ancestors 'self'</i> directive which supersedes X-Frame-Options in modern browsers.	

V-10 Missing X-Content-Type-Options Header		LOW
OWASP Category	A05:2021 – Security Misconfiguration	
Affected URL	Site-wide (all responses)	
Description	The <i>X-Content-Type-Options: nosniff</i> header is absent. Without it, older browsers (and some modern ones) may perform MIME-type sniffing, potentially interpreting a response with an incorrect Content-Type as executable script.	
Business Impact	Low standalone risk, but can be combined with file upload or XSS vulnerabilities to execute malicious content that the server intended to serve as a benign file type.	
Remediation	Add <i>X-Content-Type-Options: nosniff</i> globally in the nginx or application configuration. This is a one-line change with no functional side-effects.	

5. Risk Matrix

The matrix below maps each finding by **Likelihood** (how easy it is to exploit) and **Impact** (the potential business damage). Findings in the top-right cell require immediate remediation.

Impact → Likelihood ↓	LOW Impact	MEDIUM Impact	HIGH Impact
HIGH Likelihood		V-10	V-01 V-02 V-03 V-04 V-05 V-06 V-07
MED Likelihood		V-08 V-09	
LOW Likelihood			

6. Remediation Roadmap

Remediation actions are grouped by priority to help development and operations teams plan their sprint work effectively.

■ Phase 1 — Immediate (0–2 Weeks)

- Upgrade PHP to a supported version (8.2+) — EOL software is the root cause of many findings.
- Enable HTTPS / TLS site-wide and enforce with HSTS; mark session cookies Secure.
- Replace all dynamic SQL with parameterised queries to eliminate SQL Injection (V-01).
- Implement output encoding on all user-supplied content rendered to HTML (V-02).
- Disable PHP allow_url_include and allow_url_fopen; restrict file-path inputs (V-03, V-04).

■ Phase 2 — Short-Term (2–4 Weeks)

- Implement CSRF tokens on all state-changing forms (V-05).
- Deploy a Content Security Policy header — start in report-only mode (V-07).
- Add X-Frame-Options: SAMEORIGIN and X-Content-Type-Options: nosniff headers (V-09, V-10).

■ Phase 3 — Ongoing Hardening

- Suppress server version information in nginx (server_tokens off) and PHP (expose_php=Off) (V-08).
- Introduce a Web Application Firewall (WAF) for layered defence.
- Schedule quarterly vulnerability assessments and annual penetration tests.
- Adopt a Secure SDLC: developer security training, code review gates, SAST/DAST in CI/CD.

7. Tools & References

Tool / Resource	Purpose
OWASP ZAP (Passive Scan)	Passive header and vulnerability scanning
Browser DevTools (Network Tab)	HTTP header inspection, cookie analysis
Nmap	Port discovery and service version detection
WHOIS / nslookup / dig	Domain and DNS reconnaissance
OWASP WSTG	Methodology and test case reference
Acunetix Published Scan Reports	Evidence baseline for testphp.vulnweb.com
SecurityHeaders.io	HTTP security header grading
ImmuniWeb Website Security Test	Automated header and software analysis

8. Conclusion

The passive assessment of testphp.vulnweb.com revealed a website with a **critical overall security posture**. Ten vulnerabilities were identified, with seven rated High severity. The most dangerous — SQL Injection, Stored XSS, and File Inclusion — could each independently lead to a complete compromise of user data or the server itself in a real production environment.

The good news for any real business in a similar position is that the majority of these issues are well-understood and have well-documented fixes. Upgrading to a supported PHP version, enforcing HTTPS, using parameterised database queries, and deploying basic security headers would eliminate or significantly mitigate the most critical risks within a few weeks of focused development effort.

We recommend the client adopts the three-phase remediation roadmap outlined in Section 6 and schedules a follow-up validation assessment after Phase 1 is complete to confirm findings have been resolved.

Report prepared by **Future Interns** — **Cyber Security Task 1 (2026)** • Generated 27 March 2026 • Ethical read-only assessment — no systems were harmed in the production of this report.